01 / RECOMMENDED APPROACH

Implementation plan

Build one complete, enforceable governance workflow before expanding to more agents, clouds or dashboards.

The proposed solution

Create a shared governance control plane with distributed enforcement at agent-to-tool and agent-to-agent boundaries. The control plane holds ownership, approved purpose, lifecycle, policy versions and approvals. The runtime enforcement path authenticates callers, checks current authority, constrains data sharing and records evidence before protected tools execute. Cloud and identity integrations sit behind adapters.

A concrete first use case

A bank service employee asks the coordinator to summarize synthetic case CASE-104. The coordinator delegates knowledge retrieval, obtains masked case information and drafts a status update. Reading approved knowledge succeeds. Sending restricted case fields to an unapproved external endpoint fails. Changing the case status pauses for a separate reviewer. After approval, the exact change is rechecked, executed once and linked to the original request.

Build first	Why this order matters
1. Registry + identity + gateway + one policy	Establish a trustworthy, deny-by-default path before adding automation.
2. Allowed and denied tool calls + evidence	Prove both successful use and prevention of unauthorized side effects.
3. Delegation + approvals + data rules	Demonstrate the agent-specific governance capabilities from the context.
4. Dashboard + suspension + failure tests	Make the controls observable and usable by owners and operators.
5. Portability and scale validation	Expand only after the core security behavior is repeatable.

Working assumptions

Planning baseline: three contributors, approximately four weeks, container-capable development machines, synthetic data and permission to build a local demo. These are estimation assumptions, not facts supplied by you. The plan remains useful for a shorter event by reducing integrations while keeping one full governance loop.

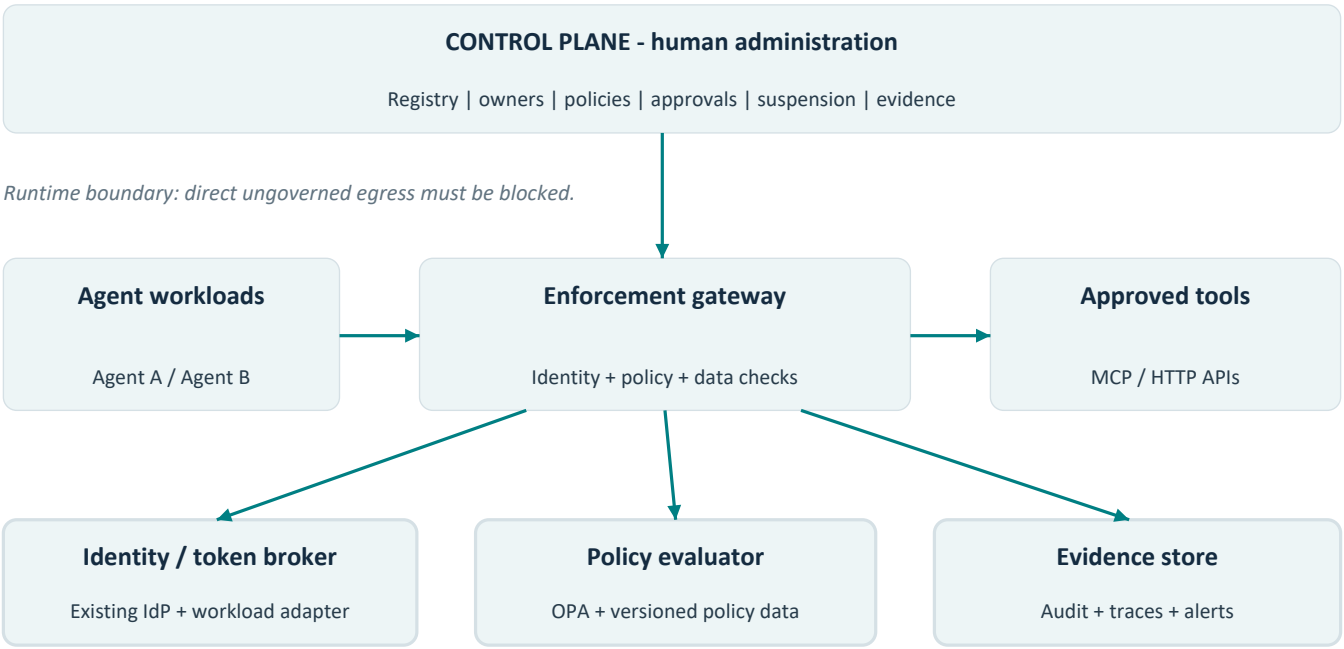
Recommended boundary

Use existing identity and policy components where practical. Write the agent-specific registry, task/delegation model, enforcement integration and evidence views. Avoid building a new identity provider or making an LLM the final authority for access.

02 / ARCHITECTURE

Control and enforcement

One management model can serve several environments; protected actions must pass through a trusted enforcement point.



Logical architecture. MVP may combine services; cloud-specific adapters remain replaceable.

Component responsibilities

Control API and portal. Manage agents, owners, tool registrations, policy releases, approvals and emergency suspension. Enforce separate owner, policy-reviewer, approver, operator and auditor roles.

Gateway and adapters. Normalize MCP/HTTP requests; verify caller identity; fetch trusted context; invoke policy evaluation; apply obligations; call the target with destination-specific credentials; inspect responses.

Identity and grants. Bind workload credentials to an approved agent version and environment. Issue short-lived task grants only after authorization. Keep downstream service secrets outside model prompts.

Policy and evidence. Evaluate explicit rules using trusted attributes. Record decisions separately from sampled performance traces; correlate both through task and request IDs.

Preventing bypass

For the demo, run agents on an isolated container network with only the gateway exposed to tools. For a cluster deployment, require an egress-enforcing networking implementation and test its behavior [T8]. Restrict model endpoints, DNS, redirects and cloud metadata access too. Tool servers should accept only authorized gateway or workload identities. An SDK alone cannot prevent a compromised agent from bypassing its checks.

The picture is logical, not a requirement for seven separate deployments. Start with one control API, one gateway, one policy engine, an identity service, a database and demo tools. Separate services only for trust, scale or ownership needs.

03 / IDENTITY AND LIFECYCLE

Know which agent is acting

Maintain a durable governance identity and independently verify the runtime instance using it.

Registry and ownership

Store a stable agent_id, version, artifact digest, business purpose, allowed task types, accountable owner reference, backup owner/team, risk tier, environment, review date and state. Keep deployments and runtime instances separate from the logical agent. A new model, prompt, tool set or artifact version should trigger a risk-based review when it materially changes behavior.

State	Entry and runtime behavior
Draft / submitted	Owner provides metadata and requested tools. No protected runtime access.
Approved / active	Reviewer accepts scope; deployment binding is verified before activation. Only active records can obtain task grants.
Suspended	Operator records a reason; new calls and grant issuance are denied. Revoke active grants and cancel work where supported.
Retired	Disable deployment bindings and credentials; preserve evidence according to approved retention. Reactivation requires a new approval.

Authentication and temporary authorization

Use one development OIDC/OAuth identity service for people and agent clients. A per-agent client credential stored outside the agent process can bootstrap a local demo, but it is not proof of production workload attestation. In a pilot, prefer platform workload federation or SPIFFE/SPIRE where it fits the hosting environment; SPIFFE provides a portable workload identity model [T3]. Do not conflate a human login token with a workload identity.

The gateway validates trusted issuer, signature, allowed algorithm, audience, expiry and the mapped agent/environment. The task service creates a grant containing the approved action/resource envelope, requester, purpose_id, run_id, expiry and grant version. Suggested demo grant lifetime: five minutes, with current suspension and revocation state checked before every protected call.

Implementation details that matter

Never trust an agent_id, owner, tenant, clearance or purpose supplied in free text. Resolve these from verified credentials and server-side records. The effective permission is the intersection of organizational policy, agent approval, requester authority where applicable, tool restrictions and the task grant. Scheduled autonomous jobs use an explicitly authorized service mandate; they must not invent a human requester.

Owner removal triggers reassignment review and suspension after a configured grace period. Separate tenant and environment identifiers must appear in storage keys, policy inputs and authorization checks. Demo defaults: one tenant and one environment; cross-tenant denial tests still apply.

04 / AUTHORIZATION LOGIC

Authorize the actual action

A structured task purpose helps express intent; deterministic rules decide whether a concrete action is allowed.

Per-call evaluation sequence

1. Authenticate and normalize. Verify the identity and registered deployment. Validate the tool name, schema, target resource and canonical arguments; reject unknown or oversized inputs.

2. Resolve trusted context. Load active agent state, approved purpose, task grant, requester entitlements, tool classification, destination, policy version and current revocation version.

3. Apply mandatory rules. Reject unknown identities, wrong audiences, expired grants, forbidden destinations, missing attributes and disallowed actions. Explicit denials take precedence.

4. Evaluate conditional access. Allow, deny or require approval. Return a decision_id, reason codes, expiry and obligations such as field masking, output size limit or one-use approval.

5. Enforce and execute. The gateway must satisfy every obligation, persist an execution-intent record, recheck freshness and then invoke the target. Unsupported obligations cause denial.

6. Inspect and record. Validate the response, apply data controls, record the outcome and release only authorized output. Denials should explain the rule without exposing protected data.

Example request	Proposed outcome
Active knowledge agent reads approved policy article for a registered case task	Allow read, enforce record filter and output limit; log policy version.
Agent asks for all customer records under the same general purpose	Deny: the task grant only covers the selected case and approved fields.
Coordinator delegates status update using a read-only parent grant	Deny: child permissions cannot exceed parent authority.
Approved case-update agent changes a permitted case field	Require exact-request human approval, then reauthorize before execution.

Policy implementation and limits

Use OPA for versioned authorization rules [T4]. Policy inputs come from trusted services and validated request data; labels supplied by the agent are advisory. Keep time, resource state and revocation data fresh. Do not cache approvals for writes across different requests. An optional model can suggest intent categories or flag anomalies, but cannot override a denial or grant itself access.

Approval is a condition inside otherwise valid authority, not a universal override for forbidden destinations or missing permissions.

05 / DELEGATION AND APPROVAL

Preserve authority across agents

A child agent needs its own identity and a narrower task grant, plus a verifiable link to its parent task.

Delegation contract

The parent requests delegation through the task service, identifying the child agent, task type, allowed actions/resources, data constraints and expiry. The service authenticates the parent, checks the child is approved for the task and computes the intersection of parent authority, child eligibility and current policy. It records root_run_id, parent_run_id, grant_id, parent_grant_id and delegation depth. Suggested demo limit: two delegation edges.

The child receives a new audience-bound credential or opaque grant reference bound to its verified identity. It never receives the parent's raw bearer token or broad service credentials. Expiry cannot exceed the parent's expiry; scope and destination cannot grow. Reject cycles, disallowed children and depth overruns. Revocation of a parent invalidates descendants through ancestry checks or a revocation index.

OAuth token exchange is a standards-based option where the selected identity provider supports the required flow [T2]. It does not by itself enforce our delegation rules or guarantee transitive revocation. For the MVP, server-side grant records and broker-enforced checks are a simpler, explicit implementation.

Approval that cannot be reused for a different action

Stage	Implementation requirement
Create request	Store canonical tool, operation, resource, arguments digest, agent, requester, task, policy version, resource version and expiry.
Review	Show a readable exact change, purpose, data involved and relevant rule. Reviewer must have the required role and cannot approve their own high-impact request.
Consume	Bind approval to the original digest and identity; verify current permissions, resource version, grant and lifecycle. Atomically reserve a one-use execution key.
Execute / recover	Use a downstream idempotency key or conditional write. If the result is uncertain, query the target before retrying. Mark approved, executing, succeeded, failed or unknown.

Suspension and cancellation

Before each side effect, check the latest revocation state and all grant ancestors. Stop dispatching new work after suspension; expire queued approvals and signal cancellation to active tasks. A kill switch cannot reverse a completed external action or guarantee cancellation of a tool that does not support it. The operator must see in-flight and uncertain operations for reconciliation.

Suggested approval expiry: ten minutes, constrained by a still-valid task grant at execution. If the task grant expires while awaiting approval, restart authorization; do not silently extend it.

06 / TOOLS AND DATA

Govern the full data path

Inspect both the call going out and the data coming back, including transfers to another agent or a model endpoint.

Tool onboarding

Register a stable tool_id, transport/protocol version, endpoint, schema digest, supported operations, owner, authentication method, allowed environments, data classification, side-effect level and approved destinations. Pin the tested schema/version. A changed schema, endpoint or permission model triggers review; do not let newly discovered tools become trusted automatically.

MCP adapter boundary

Implement the selected MCP SDK and protocol version behind a gateway adapter and test it against the chosen server. MCP transport authorization and tool-level authorization are separate concerns. Validate resource audiences, use separate downstream credentials and do not pass a client token through to another API [T1]. Local stdio servers require process isolation and restricted credentials; an HTTP gateway does not automatically govern them.

Data path	Proposed control
Agent to tool	Schema validation, approved record/field scope, payload size limit, destination allowlist and secret/identifier checks.
Tool to agent	Response schema check, source-based classification, field projection or masking before release. Block unknown restricted fields.
Agent to agent	Authorize recipient and task; transfer only approved fields. Carry trusted classification metadata with the task envelope.
Agent to model	Use approved model endpoints and the same data-egress boundary. Strip secrets and unneeded customer data before prompt submission.
Events and logs	Record categories, counts and opaque resource references. Exclude credentials and full prompts/responses by default.

Classification and leakage prevention

For the demo use four labels: public, internal, confidential and restricted. Source metadata and registered field definitions are authoritative; an agent cannot downgrade a label. Combine explicit field rules with bounded pattern checks for synthetic sensitive values. Unstructured text detection is imperfect, so a passing scan is not proof of safe content. Deny unknown data for restricted destinations and test common bypasses.

Treat retrieved instructions and tool output as untrusted content. Content cannot alter gateway policy, choose arbitrary destinations or grant privileges. Prevent SSRF through endpoint registration, URL normalization, resolved-address checks and redirect validation [T1]. Do not release streamed restricted responses before the relevant data check has completed; disable streaming for those demo operations.

Mask sensitive policy inputs before logs leave the policy engine [T4b]. Apply telemetry minimization at instrumentation and collector stages [T5]. These controls reduce exposure; data retention and residency still need client decisions.

07 / BUILDING BLOCKS

Technology and integration plan

These are replaceable recommendations for a small team, not requirements from the problem statement.

Layer	Recommended first choice	Reason / alternative
Control API and gateway	Python + FastAPI [T6]	Typed request validation and API documentation. Java/Spring or TypeScript can replace it if the team is stronger there.
Management UI	React + TypeScript	Inventory, approval queue, decision timeline and suspension controls. A thin UI is sufficient for the first end-to-end slice.
Persistent records	PostgreSQL	Transactional lifecycle, approvals, grants and audit outbox. Add a cache only when measured demand requires it.
Identity	Keycloak for local development; enterprise IdP adapter for pilot [T7]	Reuse OIDC/OAuth. Validate the chosen provider's exact token flows in an integration spike.
Policy	OPA with versioned policies [T4]	Keep authorization rules separate from business logic; test and review releases.
Observability	OpenTelemetry + selected trace/metrics viewer [T5]	Correlation and operational measurements; durable audit events remain a separate record.
Packaging	Docker Compose locally [T9]; Kubernetes if required	Portable demo on one host. Production cloud adapters and network controls need separate validation.

Build versus integrate

Build the registry, task/grant service, adapters, approvals and evidence views. Integrate identity, policy evaluation, storage and telemetry. Start with synthetic tools, then replace one adapter with a real sandbox integration.

Suggested repository layout

apps/control-api and apps/portal; services/gateway; packages/contracts and packages/agent-sdk; policies; adapters/identity and adapters/tools; demo/agents and demo/tools; tests/security and tests/integration; deploy; docs/handbook. This is a proposed layout, not an existing code repository.

Contract-first integration

Version the schemas. Authenticate callers and derive trusted owner/tenant context server-side. Use idempotency keys for mutations, record versions for edits and authorization on every lookup. Return reason codes and correlation IDs without leaking secrets.

Do not finalize dependency versions from this document. Select compatible, supported versions at implementation kickoff, lock them, scan dependencies and record tested MCP/identity interoperability.

08 / DELIVERY ROADMAP

A staged build with exit gates

Illustrative effort: 15-20 working days for three contributors, subject to scope, skills, access and integration findings.

Stage / indicative time	Work and output	Exit gate
0. Clarify + spike 1-2 days	Confirm inputs; map a single workflow; test identity issuance, gateway routing and one tool; agree threat model.	Authenticated call succeeds; bypass fails; unknowns logged.
1. Thin end-to-end slice 3-4 days	Registry, owner/state validation, one protected tool, OPA decision and durable event timeline.	Unknown agent denied; allowed/denied calls proven against tool counters.
2. Agent-specific controls 4-5 days	Task grants, constrained delegation, exact-request approval, input/output field controls and suspension.	Escalation, replay, data leakage and revocation tests pass.
3. Usable demonstration 3-4 days	Portal, three agents, two tools, seeded scenarios, automated reset and evidence export.	A reviewer can run the happy path and inspect every denied attempt.
4. Reliability + handover 4-5 days	Failure injection, load baseline, dependency review, recovery procedure, demo recording and handbook update.	Acceptance matrix has results; open gaps are disclosed.

Suggested ownership

Contributor A owns gateway, identity and tool integration. Contributor B owns registry, grants, policies, approvals and data storage. Contributor C owns the portal, demo agents, acceptance scenarios and documentation. All three review security boundaries; a separate reviewer approves policy releases and high-impact test actions where feasible.

If the event is only a few days

Reduce to one coordinator, one child agent, one read/write mock service and a small portal. Keep identity checks, purpose-scoped grants, deny/allow enforcement, one approval, one delegation denial, suspension and connected evidence. Defer multiple clouds, automatic discovery, anomaly models and infrastructure federation. A working complete path is a stronger demonstration than many simulated controls.

Enterprise pilot after the demo

Choose one business team and one sandbox environment. Integrate its identity provider, ownership directory and one real tool. Validate networking, secret management, data residency, independent audit storage, backup/restore, support ownership and the required load. Run in observation mode only for comparison, clearly marked as non-enforcing, before a controlled rollout of blocking policies.

No budget or delivery date can be committed from the current inputs. Estimate labor, identity/tool integration effort, hosting, model usage, storage and operational support after questions Q01-Q08 are answered.

09 / VERIFICATION

Tests that prove the controls

The table is a proposed acceptance suite. No test results or performance claims exist yet.

Test	Expected result and evidence	Maps to
T01 Register / activate	Missing owner or purpose prevents activation; approved deployment binds to the correct agent version.	R01-R04
T02 Identity abuse	Unknown, expired, wrong-audience and forged credentials fail before any tool execution.	R02, R07
T03 Purpose and scope	Correct case read allowed; different case, field or action denied. Tool counter verifies denial.	R05-R07
T04 Delegation	Valid child succeeds; scope growth, wrong child, cycles and excess depth fail. Full ancestry recorded.	R08
T05 Data boundary	Restricted synthetic field blocked/masked as designed on request, response and child transfer. Logs contain no seeded secret.	R09, R10
T06 Approval integrity	Self-approval, expired approval, changed arguments and reused approval fail. Concurrent retries do not duplicate a write.	R06, R07
T07 Revocation	Suspend root agent and parent grant during queued/active work. New child/tool calls fail within measured target.	R04, R06
T08 Bypass and SSRF	Direct tool egress, unregistered endpoints, unsafe redirects and metadata-service destinations are blocked.	R07, R09
T09 Audit completeness	Reconcile attempted calls, decisions, executions and outcomes; duplicates handled; missing completion flagged.	R10, R11
T10 Dependency outage	Policy/identity/state unavailable denies protected calls; audit capacity failure stops dispatch; UI shows degraded state.	R07, R10
T11 Isolation and roles	Wrong tenant/environment access denied; unauthorized users cannot approve, change policy or suspend agents.	R01-R04
T12 Portability / load	Redeploy on a second approved runtime without changing policy semantics; benchmark the agreed traffic profile.	R12

Evidence to retain

For each test keep the seed data version, policy version, artifact digest, environment, request/decision IDs, sanitized event export, tool-side execution count and observed result. Use a deterministic scripted agent path for repeatability; run an LLM-driven variation separately to show agent behavior without making acceptance dependent on model randomness.

A passing unit test is insufficient for a boundary control. Identity, gateway, network, downstream side effects and evidence capture must also be tested together.

10 / RELIABILITY AND CAPACITY

Operate safely under failure

Suggested targets are starting points for discussion and must be measured in the selected environment.

Concern	Proposed first-version behavior / target
Authorization latency	Measure gateway + policy + state-check overhead separately from model/tool latency. Start with a p95 target below 100 ms at 50 protected calls/second in the demo environment.
Suspension propagation	Target at most 5 seconds for new calls on the demo path. Check authoritative revocation on each dispatch; reject stale/unavailable state.
Audit durability	Persist an execution-intent/outbox record before tool dispatch. Deliver at least once with unique event IDs; reconcile incomplete operations.
Unavailable dependencies	Fail closed for protected calls when identity, policy, grants or required audit persistence cannot be checked. Keep the UI readable with a degraded-state notice.
Data volume and quotas	Set per-agent request, concurrency, output-byte and task-depth limits. Apply shared task budgets to descendant calls; cancel runaway loops.
Production availability	Client must define service level, recovery time and recovery point. Add redundant gateways, database recovery and tested regional boundaries before production use.

An explicit sizing example, not a forecast

If 5,000 registered agents have 10% active at once and each active agent makes 0.2 protected calls/second, steady traffic is 100 calls/second. A 5x burst gives 500 calls/second. At four audit events/call and 2 KB/event, steady raw event storage is approximately 69.1 GB/day, or 2.07 TB for 30 days before indexes, replicas and compression. Agent count alone cannot determine infrastructure size.

Failure handling and change safety

Use transactional outbox delivery so a successful control-plane update is not lost before audit publication. A remote tool side effect cannot be made atomic with the local database: record pending execution, pass an idempotency key where supported and reconcile unknown outcomes. Do not retry uncertain writes blindly. Use signed or integrity-checked policy releases, peer review, canary evaluation and rollback to a known version.

For a pilot, use separate audit write/read roles and protected append-only exports with independently controlled retention. A hash chain alone does not make a store immutable. Keep raw customer payloads out of audit logs and include configuration changes and administrator actions in evidence.

The gateway is a sensitive service because it can reach downstream tools. Limit each adapter's credentials, restrict its egress, rotate secrets and monitor privileged operations. Production security, privacy and regulatory applicability require the client's review; the framework supplies evidence, not automatic certification.

11 / INPUTS FROM YOU

Decisions needed to start

Reply by question number. If you are unsure, write "use the recommendation" and we can retain the proposed default.

Question / input needed	Recommended starting choice	Why it changes the build
Q01. What is the deadline, expected deliverable and judging rubric? Can you share the full brief or missing transcript?	Working demo + these documents + short recorded walkthrough. Treat missing requirements as unresolved.	Determines scope, proof required and whether the four-week estimate fits.
Q02. How many contributors and what are their strongest skills?	Three contributors; Python backend and TypeScript UI if skills are neutral.	Sets ownership, schedule and whether a simpler UI or different language is better.
Q03. Which banking workflow should the demo show?	Synthetic case assistance: knowledge lookup, masked case read and approval-gated status update.	Defines tools, data, permissions and the narrative for judges.
Q04. Where may it run? Any cloud, laptop, containers or Kubernetes restrictions?	Local containers first; deploy the same application to one approved sandbox next.	Determines networking controls, identity setup and required hosting access.
Q05. Is any stack or vendor mandatory? What identity provider already exists?	Vendor-neutral contracts; a local development identity service, then one enterprise adapter.	Avoids costly integration assumptions and clarifies login/workload authentication.
Q06. Which agent frameworks, MCP servers or APIs already exist? Can you provide schemas and sandbox endpoints?	Start with three small demo agents and two mock tools; then integrate one real sandbox tool.	Makes adapters concrete and confirms control over tool execution.
Q07. Can all agent traffic be routed through a gateway and direct tool egress be blocked?	Yes for the demo sandbox. Use network restrictions plus tool-side identity checks.	If no, describe coverage as monitoring/partial enforcement rather than complete control.
Q08. What data may be used and which model endpoints are approved?	Synthetic data only, one approved model endpoint or deterministic mock, no production credentials.	Determines data policies, model integration and approvals for using real data.

These are the highest-priority inputs. You do not need to know every technical answer; deadline, team, permitted environment and any supplied hackathon rules are the best starting points.

12 / REMAINING INPUTS

Governance choices and references

These decisions can follow the first scope discussion, but must be resolved before a real pilot.

Question	Suggested default / requested input
Q09. Who owns agents, approves access and handles incidents?	Separate owner, reviewer and operator roles; a different reviewer approves high-impact actions. Supply role names, not personal secrets.
Q10. Which actions are prohibited or require approval?	Allow scoped reads; approve case writes; deny payments, bulk exports and unapproved destinations in the demo.
Q11. What scale and response times matter?	Provide registered agents, active tasks, calls/second, bursts and payload sizes. Use page 10 values only as demo hypotheses.
Q12. What retention, region and audit rules apply?	Synthetic demo logs for 30 days, configurable. Client specifies pilot retention, location, access and deletion rules.
Q13. What tools, accounts and budget are available?	Provide repository access, permitted runtime, sandbox identity/tool details and model usage limits through approved channels. Do not paste passwords or tokens.
Q14. What should the final handover contain?	Runnable source, repeatable demo, automated tests, deployment guide and this versioned handbook updated to the implemented state.

Technical reference register

Official references checked 09 October 2026. They support the mechanisms; the architecture, defaults and estimates are project recommendations. Validate compatibility at kickoff.

[T1] MCP: Authorization and Security Best Practices. Audience separation, token handling and endpoint security.

[T2] IETF: RFC 8693 - OAuth 2.0 Token Exchange. Delegation/token exchange building block.

[T3] SPIFFE: SPIFFE Concepts. Portable workload identities and identity documents.

[T4 / T4b] Open Policy Agent: Documentation and Decision Logs. Policy evaluation and sensitive-field masking.

[T5] OpenTelemetry: Handling sensitive data. Telemetry minimization.

[T6 / T7] FastAPI; Keycloak OIDC endpoints. Candidate application and identity components.

[T8 / T9] Kubernetes Network Policies; Docker Compose. Deployment and network boundary references.

Context sources [S1, S2] and their limitations are recorded on page 4 of the problem statement. Answers to Q01-Q14 should become dated decisions in the handbook, with unresolved assumptions preserved.